FotoWare on-premises to Veloz migration — log analysis

Prepared 26 October 2026

Main finding

The strongest explanation is a file-specific operation that never finishes, leaving FWCopy waiting for a batch to complete. The first file to investigate is:

```
D:\DigitalAssets\Images\Historic\Oil & Gas\International\Colombia\LCI 2017\
```

This is a strong candidate, not a proven defective file. The logs do not record the start of each file operation or associate upload chunks with file paths. They cannot establish whether the wait occurs during local file access, metadata extraction, a network request, or client task completion. They also cannot exclude a batch bookkeeping defect. A controlled reproduction is needed.

Scope and limits

Read all 88 numbered logs in Archive.zip plus the additional `log.txt` : 217,180 lines in 89 logs. Ignored ZIP __MACOSX resource-fork entries. Correlated with FileList (1).txt recovered from the earlier chat. Analyzed 28 file-processing runs, three earlier generic operation errors, and three explicit history resets. Dates span 2 September–18 October 2026. Timestamps below are verbatim log times; the files do not identify their time zone.

The requested /mnt/data path was not present on this desktop. The actual attached Archive.zip and FileList were recovered through the referenced chat. No source files, migration history, or destination assets were changed.

1. The same one-file gap repeats in every incomplete run

All 22 incomplete processing runs have exactly one fewer logged terminal file outcome than the declared size of the final batch. A terminal outcome here means either 'File successfully transferred' or 'has not changed, skipping copy'. None of these 22 runs logs completion of that final batch or 'All batches have been processed'. This is substantially stronger evidence than the final filename alone.

Run	Total transfer messages	Final batch	Declared files	Recorded outcomes
`log.11.txt`	12,895	65/214	198	197
`log.14.txt`	234	2/153	256	255
`log.17–65.txt` *	39 each	1/144–149	302	301
`log.83.txt`	12,430	63/210	195	194
`log.86.txt`	12,382	63/208	198	197
Additional `log.txt`	12,467	64/210	197	196

*Seventeen processing logs: 17, 20, 23, 26, 29, 32, 35, 38, 41, 44, 47, 50, 53, 56, 59, 62, 65. Other numbered files in this range mainly contain housekeeping records.

The seventeen retries each log 39 transfers plus 262 unchanged files. The first fifteen retransmit the same 39 paths; the final two have a slightly shifted set. This suggests repeated work without durable progress beyond the blocked batch. Two workers are configured, which is consistent with one operation remaining pending while the other processes the remainder. That mechanism remains an inference.

Batch denominators change with advertised processing capacity. Different final batch numbers therefore do not establish different root causes.

2. FileList narrows the candidate to `_D8A3296.jpg`

FileList lines 3882–3884 contain, in order:

```
_D8A3295.jpg
_D8A3296.jpg
_D8A3297.jpg
```

The middle file has no success or unchanged record anywhere in the 89 logs. Both neighbors repeatedly succeed:

- `log.11.txt:51297` — 2026-09-02 13:56:12.5041 — `_D8A3295.jpg`
- `log.11.txt:51306` — 2026-09-02 13:56:14.5400 — `_D8A3297.jpg`
- `log.17.txt:342` — 2026-09-03 09:31:47.7640 — `_D8A3295.jpg`
- `log.17.txt:355` — 2026-09-03 09:31:49.8274 — `_D8A3297.jpg`
- `log.83.txt:50384` — 2026-10-15 14:51:07.2784 — `_D8A3295.jpg`
- `log.83.txt:50394` — 2026-10-15 14:51:08.8607 — `_D8A3297.jpg`
- `log.86.txt:50384` — 2026-10-18 17:02:05.1085 — `_D8A3295.jpg`
- `log.86.txt:50394` — 2026-10-18 17:02:06.6098 — `_D8A3297.jpg`
- `added log.txt:50395` — 2026-10-18 18:35:23.1546 — `_D8A3295.jpg`
- `added log.txt:50405` — 2026-10-18 18:35:24.8538 — `_D8A3297.jpg`

The October 18 earlier run stops immediately after `_D8A3297.jpg`. Other runs continue through later files in the same directory before their incomplete batch stops. With concurrent processing, the last successful filename need not be the blocked filename. Do not infer that `_D8A3297.jpg` or the various later last-success filenames are corrupt.

Absence alone would not prove an error: a file might have been excluded or absent at run time. Its position among repeatedly transferred neighbors, combined with the invariant one-file gap, makes `_D8A3296.jpg` the leading test candidate. The logs lack a complete per-batch input manifest, so its actual inclusion in each batch is not proven.

3. The additional log continues the same pattern

The added log begins on 2026-10-18 at 17:24:20.2428 and explicitly archives `log.88.txt`, connecting it to the ZIP chronology. It classifies 41,037 files, transfers 12,467, and completes 63 batches. At 18:34:10.2003 it begins batch 64 with 197 images (lines 49550–49552). Only 196 transfer outcomes follow. Its last entry is a successful transfer of `LCI 2017\_DSC2203.jpg` at 18:37:25.9710, line 51091.

It contains no Error, Warn, or Fatal severity entries, no completion of batch 64, and no normal all-batches completion. Its transferred paths are all present among earlier success records. It reinforces the recurring incomplete-batch pattern but does not independently prove that the process was hung rather than stopped or that the log capture was complete.

4. History resets restart work without removing the

blocker

Explicit resets occur at:

- `log.82.txt:2` — 2026-10-15 13:34:28.5701
- `log.85.txt:2` — 2026-10-18 15:48:58.9907
- `log.88.txt:2` — 2026-10-18 17:24:19.5025

The following large runs start uploading again without the earlier global prefilter exclusions. Each eventually reaches an incomplete batch in the same LCI 2017 area.

Across all logs there are 51,076 successful-transfer messages for only 12,937 distinct case-insensitive full paths; five paths belong to the desktop test, leaving 12,932 D:\DigitalAssets paths. This measures logged transfer coverage, not verified Veloz asset creation. Repeated transfer messages do not prove duplicate destination assets; destination overwrite/deduplication behavior is not established here.

Global 'already-uploaded' filtering and per-file 'unchanged' checks also disagree operationally. For example, `log.17.txt:19` accepts all 302 batch files, but 262 later skip as unchanged. The next retry still reports the same 12,415 global exclusions and repeats 39 transfers. Different filter semantics or incomplete state persistence may explain this. Ask FotoWare to verify when history is committed and how these two checks differ before calling the history corrupt.

5. Other findings and alternatives

- Only three explicit Error entries occur: `log.1.txt:2` at September 2 12:23:12.6603; `log.3.txt:2` at 12:25:18.6869; `log.5.txt:2` at 12:26:07.0146. All say only 'Error running FWCopy operation'. They contain no exception detail. The five-image test then succeeds in `log.8.txt`, so these early errors are not enough to explain the later deterministic stalls.
- All 276 recorded capacity responses report `FWP Status=OK`; every returned capacity category is positive. The final added-log response advertises `imageCapacity=197`. There is no logged capacity exhaustion, HTTP authorization failure, timeout, or retry error. This makes those explanations weaker, although capacity endpoint success does not prove every upload request succeeds.
- All 28 processing runs clamp `--post-processing-interval=0` to 1. This is a configuration issue to correct, but successful small runs use it too, so it is not established as the primary cause.
- The five-image test completes; the eight-file unchanged test in `log.68.txt` completes;

four zero-work runs complete. Bulk processing and resumptions are where progress stops.
- Source scope changes: 42,365 initially; 40,947 by `log.65.txt`'s prefilter; 41,037 in the later reset runs. Compare job selection, source deletions and exclusion rules; the logs do not establish why the counts changed.
- Broad file selection includes non-asset material. `log.11.txt` reports 249 .db transfers and 69 extensionless paths, including .stndb files, plus .dix and XML files. Their upload success does not establish that they become useful Veloz assets. They do not explain a specifically proven type failure.

6. FileList correlation and its limits

The FileList contains 43,178 path lines, 43,173 distinct exact strings, and both files and directories. Identifying directories that are parents of other entries leaves 42,076 leaf entries; empty directories cannot be reliably separated from files by this method. These are not directly comparable to the migration input count.

Its main filename suffix counts are 35,930 .jpg, 3,884 .tif, 559 .xmp, 367 .jpeg, 216 .ix, 123 .psd, 118 .gif, 83 .xml, 81 .doc, 81 .nef and 78 .pdf. Counts retain duplicate listing lines. Dotted directory names were excluded when identifiable as directories.

OEM cp850/cp437 decoding matches accented log paths better than cp1252; the candidate filename is ASCII and unaffected. Case-insensitive exact-path matching finds 12,550 of the 12,937 distinct logged successful paths in the FileList. Unmatched paths include hidden/system artifacts and the desktop test; inventory timing and filename encoding can also affect matching. An unmatched path is not evidence of a migration failure.

The 559 listed XMP paths warrant a metadata-specific reconciliation. The available success messages do not establish whether sidecar metadata was consumed, embedded or uploaded separately. Verify metadata as well as file presence. The 15 listed .mp4/.mov video paths all have successful-transfer records in `log.11.txt`; this does not verify playback, metadata or final ingestion.

Prioritized diagnostic steps

1. Preserve current FWCopy history, configuration, executable version and logs. Avoid another full history reset until its behavior is understood. Record the source timestamp, size and hash of `_D8A3296.jpg` and its two neighbors. Confirm the suspect still exists, is included in the current job, and is readable by the actual migration account.

2. Run a small controlled test of those three files to an agreed test destination, preserving originals. Test the suspect alone and the neighbors as controls, using supported debug logging. Compare local copy/read, full JPEG decoding and metadata extraction. Do not rewrite or strip metadata from the only original. If testing a sanitized derivative, keep it separate and compare outcomes.

3. Run the same small selection with one worker, using the supported configuration for this FWCopy build. Request per-file start/end logging, upload request identifiers, HTTP status and timeout details. If it stops, collect the process/thread dump before restarting. This distinguishes a local read/metadata wait from network or task-completion waits.

4. In a controlled copy of the job, exclude only `_D8A3296.jpg`, retain history, and test whether batches advance beyond LCI 2017. Pair this with the isolated-file test: exclusion success plus isolated reproduction would provide much stronger causal evidence. If exclusion does not help, bisect the remaining incomplete batch using a captured input manifest.

5. Give FotoWare support the one-file deficit table and numbered evidence. Ask them to inspect cancellation/timeouts, completion tracking with two workers, exception handling, history commit timing, and global-versus-per-file skip semantics for the exact FWCopy version. Correlate Veloz/FWP ingestion records at the listed timestamps; confirm the log time zone first.

6. After resolving the blocker, reconcile a stable source manifest against destination asset IDs, sizes/hashes where available, ingestion status, metadata, sidecars and playback. Separate excluded system/index files from intended assets. Decide asset selection rules and correct the interval configuration before the final bulk run.

Supporting deliverables

- run_summary.csv: one row per processing run, including final batch outcome counts.
- file_outcomes.csv: every logged transfer/unchanged outcome with source log, line and timestamp.
- filelist_correlation.csv: each inventory entry with matching transfer and unchanged logs; blank matches mean no recorded outcome, not proven failure.
- key*log*evidence.txt: timestamped numbered source excerpts, reset records and candidate neighbor context.

Public documentation lookup did not yield a sufficiently specific FWCopy troubleshooting reference to establish implementation behavior. Product-specific root-cause claims above are

explicitly inferences from the supplied logs, not documented guarantees.